Reviewer Pack — Minimal Rank of Quasi-Group Representations Bounds ACC⁰ Circ…

Entry 7fd54f81c03b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 19:06:44 UTC

Minimal Rank of Quasi-Group Representations Bounds ACC⁰ Circuit Size

Entry ID: `7fd54f81c03b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 19:06:44 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

> For all n ≤ 40, the minimal rank of a representation of a quasi-group Q is at least θ(log n) when Q is isomorphic to an ACC⁰ circuit with size O(n^(2/3)) and depth O(n^(1/3)). Furthermore, there exists a constructive mapping from an ACC⁰ circuit C to a quasi-group representation R such that the rank of R is at least θ(log |C|).

Rationale (proposer's reasoning):

> Quasi-groups have been studied extensively in abstract algebra but are rarely applied in complexity theory. This conjecture suggests that properties of quasi-group representations could provide insights into the structure and computational complexity of ACC⁰ circuits, potentially leading to new circuit lower bounds.

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fed0165bb5eea2e5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each ACC⁰ circuit C with size n ≤ 40, if the minimal rank of the corresponding quasi-group representation R satisfies |rank(R) - log(n)| ≤ 1, then support is provided for the conjecture. If any generated circuit C has a minimal rank of R with rank(R) < log(n/2) or rank(R) > log(2n), the conjecture is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal rank Quasi-group representations AND ACC⁰ circuit complexity` - `Quasi-group theory AND bounds on ACC⁰ circuit size` - `Representation mapping to quasi-group AND ACC⁰ circuits`

Top relevant hits considered: - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/2501.16039v4] Complexity of Constructing Minimal Faithful Permutation Representations for Fitting-free Groups - [http://arxiv.org/abs/1005.2254v9] On Universal Complexity Measures - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1006.5752v1] Solomon's induction in quasi-elementary groups - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    g, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return g, x, y

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist")
```

```
    else:
        return x % m

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def matrix_power(M, k):
    n = len(M)
    result = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    while k > 0:
        if k % 2 == 1:
            result = matrix_multiply(result, M)
        M = matrix_multiply(M, M)
        k //= 2
    return result

def is_quasi_group(Q):
    n = len(Q)
    for a in range(n):
        for b in range(n):
            if Q[a][b] not in range(n):
                return False
    for a in range(n):
        for b in range(n):
            found = False
            for c in range(n):
                if Q[a][c] == Q[b][c]:
                    found = True
                    break
            if not found:
                return False
    for a in range(n):
        for c in range(n):
            found = False
            for b in range(n):
                if Q[a][b] == c:
                    found = True
                    break
            if not found:
                return False
    return True

def quasi_group_representation(C):
    n = len(C)
    Q = [[0] * n for _ in range(n)]
    for a in range(n):
        for b in range(n):
            Q[a][b] = C[b][a]
    if not is_quasi_group(Q):
        return None
```

```
    return Q

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        C = [[random.randint(0, n-1) for _ in range(n)] for _ in range(n)]
        Q = quasi_group_representation(C)
        if Q is None:
            conjecture_holds = False
            counterexample = "mapping_undefined"
            break
        rank = len(Q)
        total_rank += rank
        instances_tested += 1

    mean_rank = total_rank / instances_tested
    return {
        "metric_name": "rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7a8aa2e0.py", line 129, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7a8aa2e0.py", line 115, in run_trial
    mean_rank = total_rank / instances_tested
                ~~~~~~~~~~~^~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10786
2	preregistration	ollama_remote	glm4:latest	0	0	6528
3	novelty	ollama_remote	glm4:latest	0	0	4558
4	novelty	ollama_remote	glm4:latest	0	0	6243
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16965
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13932
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19224
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12370
9	judge	ollama_remote	glm4:latest	0	0	9922

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 100529 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7fd54f81c03b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7fd54f81c03b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7fd54f81c03b.tar.gz` (if generated)